



Personalize the Shell Prompt

1. Replace the default \$ prompt with your roll number (e.g., 21000123_NAMES\$).

(5)

Syscall to Convert String to Uppercase (toupper)

2. Implement a new system call in xv6 that converts all lowercase letters in a user-provided string to uppercase, modifying the string in place.

(10)

- a) Add Syscall Number in syscall.h

```
1  #define SYS_toupper 26
```

- b) Declare and Map the Syscall in syscall.c

```
1  extern int sys_toupper(void);  
2  [SYS_toupper] sys_toupper,
```

- c) Implement the Syscall Handler in sysproc.c

```
1  int  
2  sys_toupper(void)  
3  {  
4      char *str;  
5      int len, i;  
6      if(argstr(0, &str) < 0 || argint(1, &len) < 0)  
7          return -1;  
8      //ADD YOUR CODE HERE TO CONVERT LETTERS TO UPPER CASE  
9  }
```

- d) Add Prototype in user.h

```
1  int toupper(char *str, int len);
```

- e) Add Assembly Stub in usys.S

```
1  SYSCALL(toupper)
```

- f) Create User Test Program testtoupper.c

```
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int main(void)
6  {
7      char stringArray[32] = "Write Your Full Name Here";
8      printf(1, "Before : %s\n", stringArray);
9      toupper(stringArray, strlen(stringArray));
10     printf(1, "After : %s\n", stringArray);
11     exit();
12 }
```

Display Process Memory Layout

3. Implement a new system call `int memlayout(void)`; that prints the memory map of the calling process including code, data, heap, and stack information.

Steps to Implement:

- a) Add syscall number in `syscall.h`:

```
1  #define SYS_memlayout 23
```

- b) Declare prototype in `sysproc.c`:

```
1  uint64 sys_memlayout(void);
```

- c) Add syscall entry in `syscall.c` and `usys.S`:

```
1  extern uint64 sys_memlayout(void);
2  [SYS_memlayout] sys_memlayout,
3  SYSCALL(memlayout)
```

- d) Implement syscall in `sysproc.c`:

Expected Output:

Process 5 Memory Layout:

Code/Data: 0x0000 - 0x0000f000 (15 pages)

Heap: grows from end of data to brk (sz=0x0000f000)

Stack: grows downward from 0x7fffffff

Page Fault Counter

4. In the xv6-public kernel, implement a mechanism to count the number of page faults generated by each process. Each process should maintain its own count, which can be retrieved using a new system call. At the end, write a user program to validate your implementation.

a) Add a Counter to Each Process

• File: `proc.h`

Add a new integer field in the `struct proc`:

```
1  int page_faults;
```

• File: `proc.c`

In the function `allocproc()`, initialize the counter to 0.

b) Modify the Trap Handler to Detect Page Faults

• File: `trap.c`

Inside the function `trap(struct trapframe *tf)`, add a case for page fault (`T_PGFLT = 14`):

```
1  case T_PGFLT: {
2      struct proc *p = myproc();
3      if (p == 0)
4          panic("page fault with no process");
5
6      uint va = rcr2();          // faulting address
7      va = PGROUNDDOWN(va);
8
9      if (va >= p->sz) {          // invalid address
10         cprintf("Invalid page fault at %x\n", va);
11         p->killed = 1;
12         break;
13     }
14
15     char *mem = kalloc();
16     if (mem == 0) {
17         cprintf("kalloc failed\n");
18         p->killed = 1;
19         break;
20     }
21
22     memset(mem, 0, PGSIZE);
23     mappages(p->pgdir, (char*)va, PGSIZE, V2P(mem), PTE_W
24             | PTE_U);
25
26     p->page_faults++;           // increment counter
27     break;
28 }
```

c) Enable Lazy Memory Allocation

• File: `sysproc.c`

Modify the `sys_sbrk()` system call to not allocate pages immediately:

```

1  int
2  sys_sbrk(void)
3  {
4      int n;
5      struct proc *curproc = myproc();
6
7      if (argint(0, &n) < 0)
8          return -1;
9
10     int addr = curproc->sz;
11     curproc->sz += n;    // only increase size, no
                           allocation
12
13     return addr;
14 }

```

d) Implement a System Call `getpagefaults()`

- File: `syscall.h`
Add a new syscall number.
- File: `syscall.c`
Declare the syscall.
- File: `sysproc.c`
Add the system call implementation.
- File: `usys.S`
Add the user entry.
- File: `user.h`
Add the prototype: `int getpagefaults(void);`

e) Declare and Define `mappages()` if Missing

- File: `defs.h`
Add the declaration:

```

1  int mappages(pde_t *pgdir, void *va, uint size, uint pa
    , int perm);

```

- File: `vm.c`
If not already present, add the following function:

```

1  int
2  mappages(pde_t *pgdir, void *va, uint size, uint pa,
    int perm)
3  {
4      char *a, *last;
5      pte_t *pte;
6      a = (char*)PGROUNDDOWN((uint)va);
7      last = (char*)PGROUNDDOWN(((uint)va) + size - 1);
8      for(;;){

```

```

9      if((pte = walkpgdir(pgdir, a, 1)) == 0)
10         return -1;
11         if(*pte & PTE_P)
12             panic("remap");
13         *pte = pa | perm | PTE_P;
14         if(a == last)
15             break;
16         a += PGSIZE;
17         pa += PGSIZE;
18     }
19     return 0;
20 }

```

f) Create a User Program to Test

```

1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int main(void)
6  {
7      printf(1, "Page faults before: %d\n", getpagefaults())
8      );
9
10     int pages = 4;
11     int size = pages * 4096;
12     char *mem = sbrk(size);
13
14     for (int i = 0; i < size; i += 4096)
15         mem[i] = 1; // Access each page once
16
17     printf(1, "Page faults after: %d\n", getpagefaults())
18     ;
19     exit();
20 }

```

g) Expected Output Example

```

$ faulttest
Page faults before: 2
Page faults after: 6

```

So, your process had 2 page faults before running your test code, and 4 additional page faults occurred during the page accesses (→ total 6).

Memory Allocation and Dirtying Pages

5. Base Code:

You are given the following code to start with:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main() {
6      char *p = malloc(1024);
7      if (p == NULL) {
8          perror("malloc");
9          return 1;
10     }
11
12     printf("Allocated 1KB of memory at address: %p\n", (void
13 *)p);
14     printf("Process ID: %d\n", getpid());
15     printf("Press Enter to exit...\n");
16
17     // Touch the memory to make sure it's mapped
18     p[0] = 'a';
19
20     getchar();
21     free(p);
22     return 0;
23 }
```

Your Task:

a) Modify the Allocation

Change the memory allocation from 1KB to a size that spans at least four pages, assuming a standard page size of 4KB.

b) Dirty Multiple Pages

After allocating memory, add a for loop that writes to four different pages in the allocated memory region. Each iteration should write to a distinct 4KB page. This ensures that each write operation "dirties" a new virtual memory page.

Run an Experiment to Detect Belady's Anomaly

6. **Description:** Run an experiment in xv6-public to detect Belady's Anomaly, where increasing the number of page frames causes more page faults using your FIFO implementation.

Tasks

- Write a user-space test program `belady.c` that accesses pages in the sequence:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

- Run this program twice:
 - Once with 3 page frames.
 - Once with 4 page frames.
- For each run:
 - Record the number of page faults.
 - Print the victim page sequence.
 - Identify whether Belady's anomaly occurred and explain the reason.

Steps to Implement

- a) Modify `vm.c` to track page faults and victim pages for FIFO.
- b) Write `belady.c`, a user-space program to allocate/access pages and print page faults.

Expected Output

Page faults increase when going from 3 to 4 page frames, demonstrating Belady's Anomaly.
